

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 1

ENSEMBLE LEARNING

Reading Time 18 minutes

Prior Knowledge Naive Bayes, Support Vector Machines, Decision Tree, Train and Test set.

Keywords Blending, Bagging, Stacking, Validation.

*Thus far, we have covered a variety of machine learning techniques specifically classifiers. Oftentimes, we look for various opinions and then use the majority to guide our decision in the end. Think of ratings/feedback on a product advertised online. Ensemble learning works in a similar way. Decisions from various classifiers are combined and a majority vote or weighted average is taken as the final decision. This has some benefits such as improving the accuracy, reducing the generalization error of the individual classifiers (Opitz and Maclin 169) and reducing the variance error (Ali and Pazzani 173) without compromising bias. We've touched on ensemble learning in Module 3 when we mentioned boosting, the idea of combining weak learners to make a single strong one. We look at an ensemble method, specifically **Bagging** in the next section.*

```
In [ ]: import pandas as pd
```

1. Bagging

Bagging or bootstrap aggregation is a way to reduce variance within a noisy dataset. In this process, weak classifiers are trained independently on a sample instead of the complete training set. These samples are not necessarily smaller than the training set; however, the sample size is often the same size as the training set to ensure each classifier has sufficient data to train. The sample is drawn with replacement; hence, observations can be exposed to more than one classifier and samples would have observations more than once. The weak classifiers do not necessarily have the same predictors or features. Each weak classifier will likely have different ways of predicting target classes not only on different features but also different splits on the same feature since the samples are different. This diversity among the weak learners is caused by an *inducer*. A stable inducer results in similar weak learners, which produce similar predictions that are unlikely to improve the performance of the model. It is therefore advisable to use a relatively unstable inducer that causes small perturbations to the training set. The weak learners are also homogeneous, i.e., they are of the same model type, such as all being trees for example. There is also an added benefit of parallelizing

the process since classifiers are trained on independent samples. In the end, an average for regression, or majority prediction for classification, is used as the output. This majority voting for classification is known as hard voting or majority voting.

1.1. Bagging vs Boosting

We've touched on boosting in Module 3, and we will go through it in more detail in Lessons 3 and 4 of this module. At a high level, it is worth pausing and reflecting on the differences between these two methods. Figure 1 illustrates a summary of the methodology of Bagging and Boosting. Bagging is a parallel process whereby each base learner is not dependent on another, and the dataset for each is also independently sampled. Boosting on the other hand is done sequentially such that each base learner is developed based on the previous learner. With boosting, the training dataset, or samples fed into the base learner, is derived from the predictions of the previous base learner.

Figure 1: Bagging vs Boosting

Base learner icon has been designed using resources from Flaticon.com

The downfall of an ensemble learner is the lack of interpretation of the model. It is not clear as to the decision flow through features as in a decision tree classifier since it is various classifiers that resulted in the final decision. There are methods such as **SHAP** and **LIME** that have overcome this problem and removed the black-box element of ensemble models, however accurate methods like SHAP are computationally expensive and can take a long time especially as observations that require interpreting increases. We look at another ensemble learning method called *stacking* in the next section.

2. Stacking

Figure 2: Datasets for Building a Model

To understand stacking, we first need to understand the validation data and model build set. Figure 2 shows a full dataset commonly split between a test and training set shown by A and B respectively. This is often done in Python with the `train_test_split` function from `sklearn`. We will assume we are working with a binary target with class 0 and 1. To provide some context, let's assume our full dataset has 10,000 observations. We split our data into 75/25 Train/Test split, so A and B have 2,500 and 7,500 observations respectively. The training set, B, is then further split into a 20/80 split, comprising the validation (C) and build (D) set, such that C has 1,500 observations and D has 6,000 observations. Dataset D is used to train a model, and the model's performance is assessed on C. Keep in mind that dataset D is also commonly referred to as the training set since we train our classifiers on this set; however, we use *Build* to distinguish this from

B. We can use C and D to build the model since it allows for fine-tuning a model and reduces the chances of overfitting.

With these datasets in mind, let's suppose we train 3 classifiers such as Naive Bayes (Clf1), SVM (Clf2), and a decision tree (Clf3) on D. We then apply these classifiers on the validation set, C, and obtain the following:

```
In [ ]: df = pd.read_csv("meta_dat2train.csv", index_col=False)
df
```

The last column "Y" is the actual class value. Looking at the dataset above, we can use columns Clf1 to Clf3 as inputs into another model with target variable Y. This model that we develop is called a *Meta* or *Level-1* model and the approach described is called *blending*. Datasets C and D can be randomly shuffled into different 20/80 arrangements say k . A term to describe this is called *k-fold cross-validation*. This can be shown in Figure 3. We can develop the Meta model on the k^{th} bucket and use the remaining $k-1$ buckets to train the 3 classifiers, Clf1-3. This approach over *k-folds* is known as *stacking* and is the main difference between blending. The classifiers Clf1-3 are known as base models, and this does not have to be restricted to 3, as in our example. You'll notice that 3 different families of classifiers were used, which is different from bagging. This implies that base or individual learners for stacking can be heterogeneous.

Figure 3: K-Fold Cross-Validation

Source: Gufosowa, Wikimedia Commons, September 2019, https://commons.wikimedia.org/wiki/File:K-fold_cross_validation_EN.svg.

3. Conclusion

Ensemble learning is based on the principle that many weak learners combined provide a better estimate than a single learner. This method often improves the bias-variance tradeoff problem. We've mentioned two popular ensemble learning methods, namely Bagging and Stacking. Bagging is restricted to homogeneous learners whereas stacking is not. In the next lesson, we will implement an example of each method in Python.

References

- Ali, Kamal M., and Michael J. Pazzani. "Error Reduction through Learning Multiple Descriptions." *Machine Learning*, vol. 24, 1996, pp. 173–202.
- Opitz, David W., and Richard Maclin. "Popular Ensemble Methods: An Empirical Study." *Journal of Artificial Intelligence Research*, vol. 11, 1999, pp. 169–172.

